

СОВРЕМЕННЫЕ ТЕХНОЛОГИИ РАЗРАБОТКИ WEB- ПРИЛОЖЕНИЙ

ЛЕКЦИЯ 4

Базовые сведения о CSS

CSS – назначение и история

Терминология и синтаксис CSS

Подключение CSS к веб-странице

Селекторы CSS

Наследование и каскадирование

1. CSS — назначение и история

CSS (Cascading Style Sheets, каскадные таблицы стилей) — **язык описания внешнего вида** документа, созданного с использованием языка разметки.

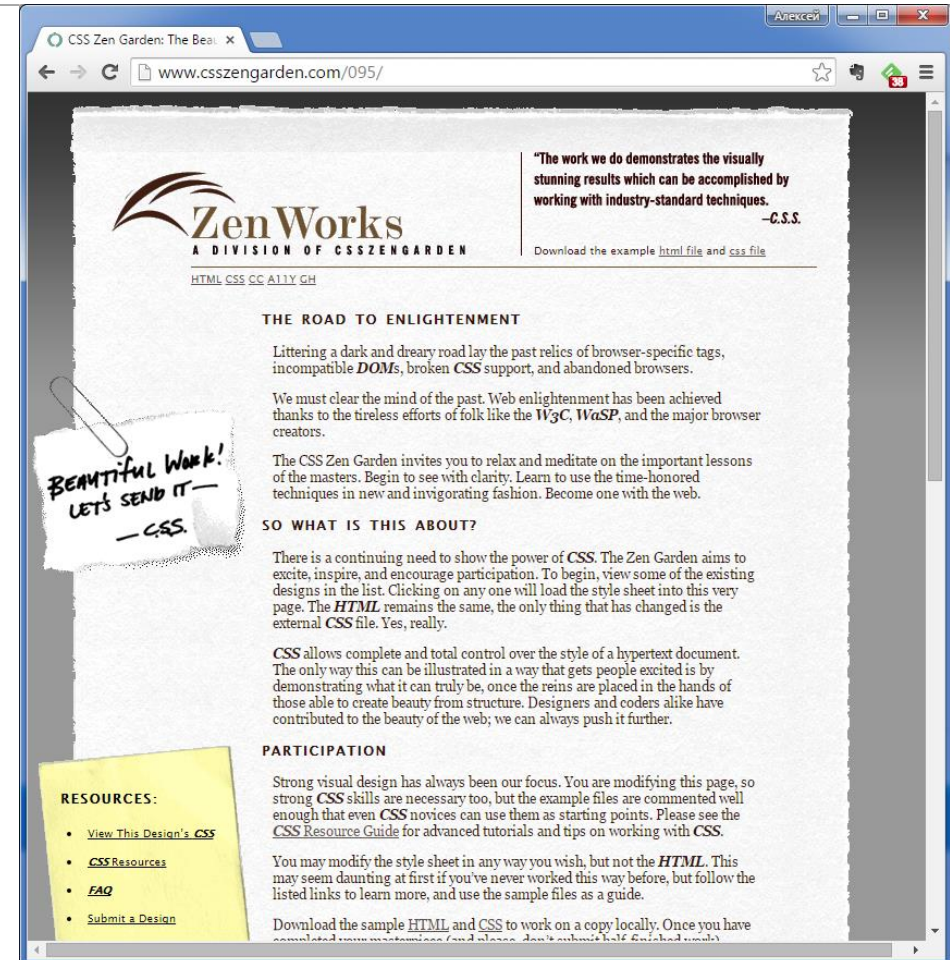
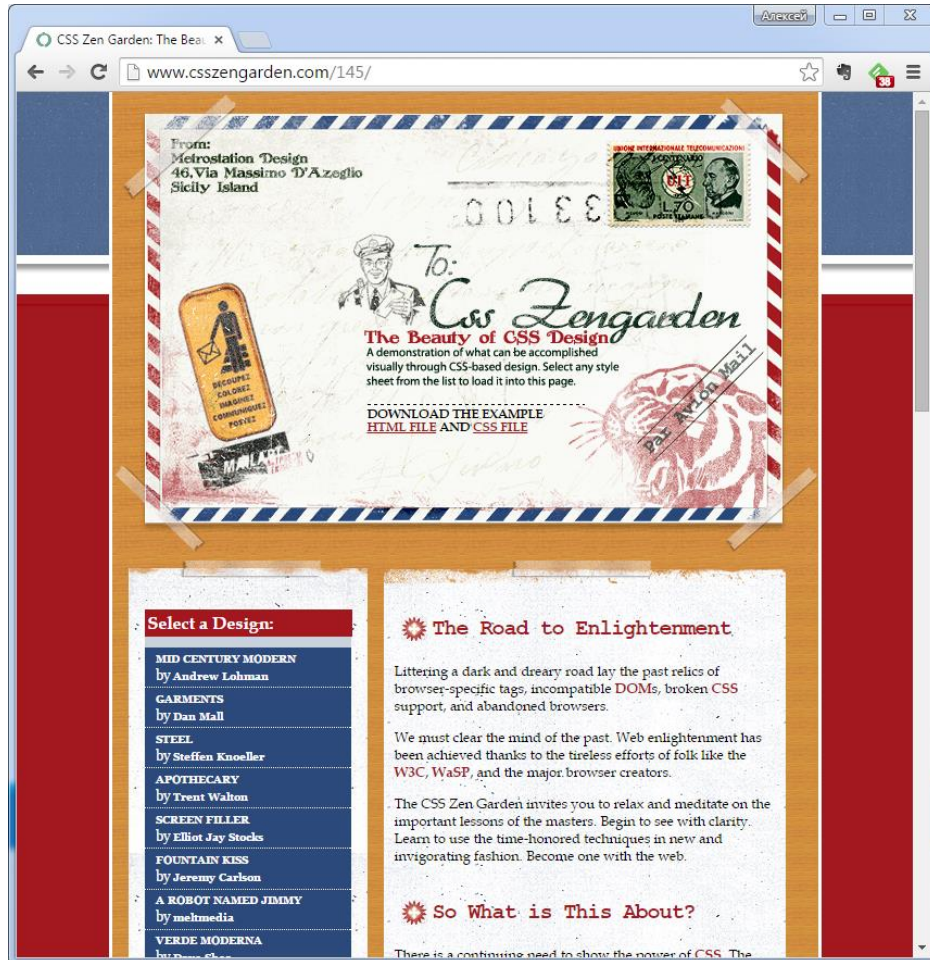
Языком разметки может быть XML, SVG, XUL, но обычно в этой роли выступает HTML.

Цель создания CSS

Цель: разделить **логическую структуру** документа (например, HTML-документа) и описание **внешнего вида**.

- + разные виды одного документа (экран, принтер, голос)
- + более «богатый» визуальный язык
- + сам документ упрощается
- + упрощается поддержка и разработка

CSS Zen Garden: www.csszengarden.com <http://www.csszengarden.com/tr/ru/>



История CSS

Первое упоминание: 1994 год

Хокон Виум Ли (Håkon Wium Lie).

Далее – затишье, к разработке подключается Берт Бос (Bert Bos).

1995 год – интерес со стороны **W3C**.



История CSS

После 1995 года: развитие CSS как языка и **создание стандарта**. Это нужно, чтобы разработчики браузеров включали поддержку возможностей CSS в свои продукты.

Версии стандарта CSS называют *уровнями* (Levels):

Уровень 1 (**CSS1**): в конце 1996 года.

Уровень 2 (**CSS2**): май 1998 года.

Уровень 2, ревизия 1 (**CSS2.1**): июнь 2011 года.

Уровень 3 (**CSS3**): разрабатываемая версия, но многие браузеры поддерживают его возможности.

Уровень 4 (CSS4): Разрабатывается W3C с 29 сентября 2011 года (working draft).

CSS3

Taxonomy & Status (Dezember 2016)

- W3C Recommendation
- Candidate Recommendation
- Last Call
- Working Draft
- Obsolete or inactive



CSS и браузеры

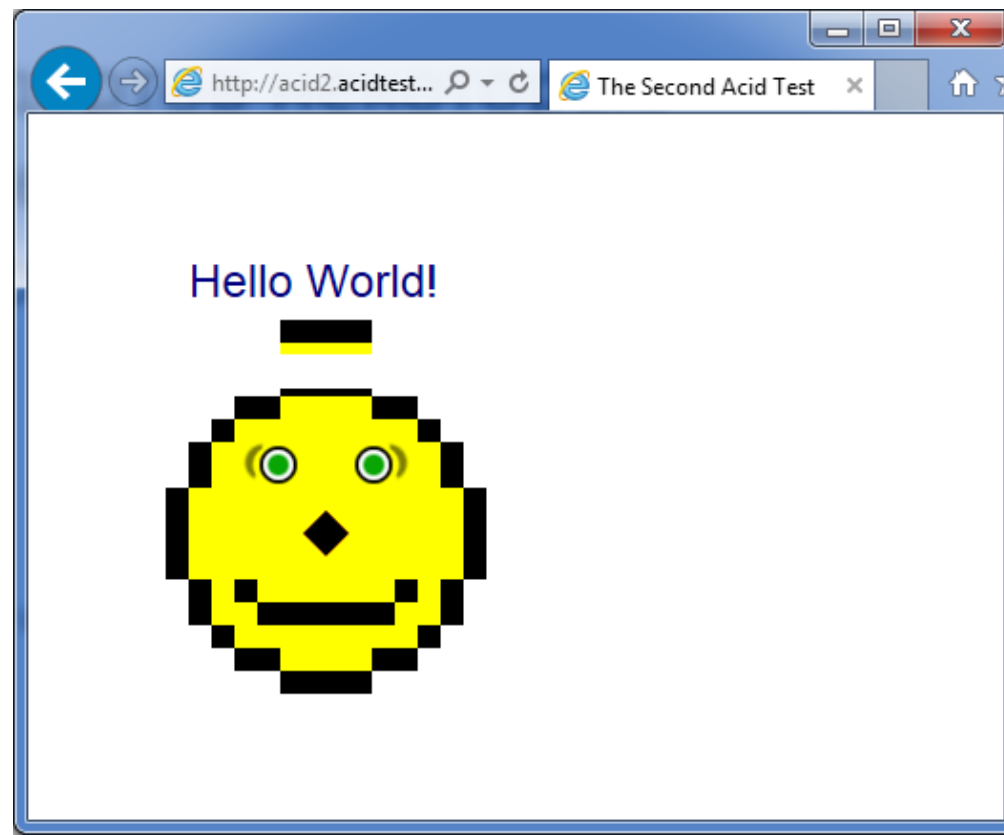
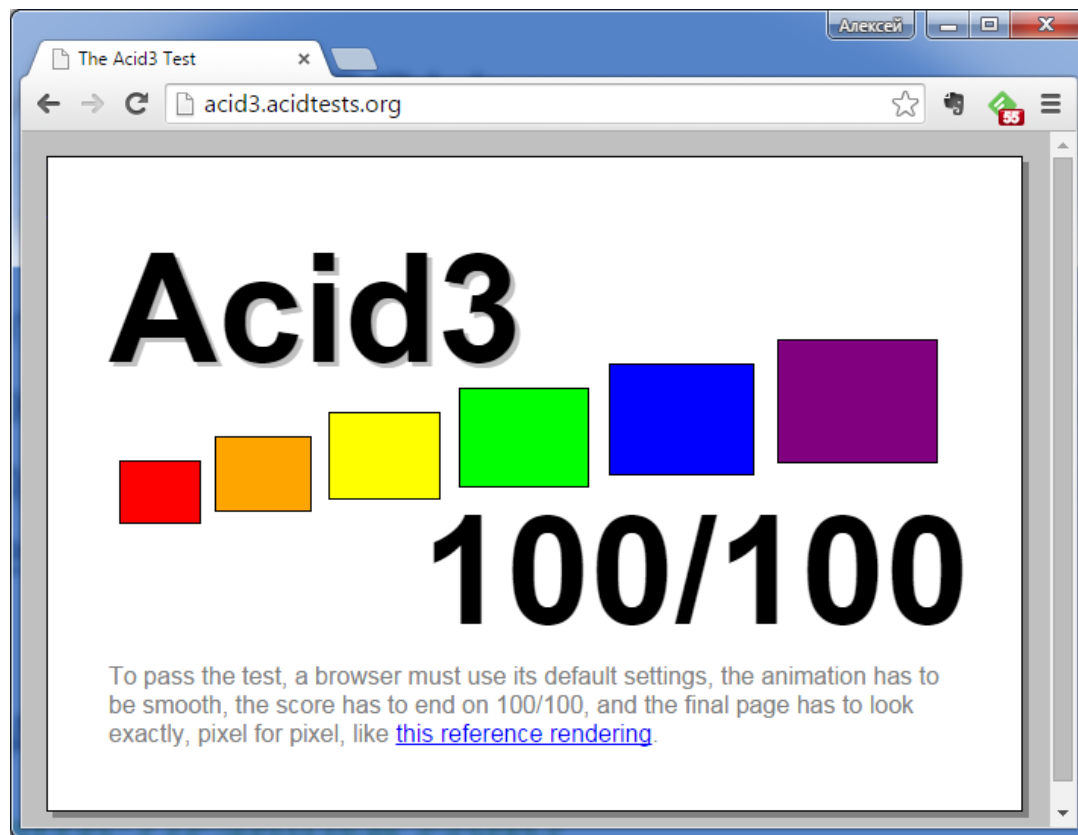
Тут не всё гладко!

Несмотря на наличие стандартов, некоторые возможности CSS (1, 2, 3) не поддерживаются вообще или поддерживаются «по-особенному» в зависимости от браузера и его версии.

Тесты для браузеров: Acid1, Acid2, Acid3.

Сайт: <http://caniuse.com/>

CSS и браузеры



(<http://jigsaw.w3.org/css-validator>)



CSS Validation Service

Проверка каскадных таблиц стилей (CSS) и документов (X)HTML с таблицами стилей

Проверить URI

Проверить загруженный файл

Проверить набранный текст

Проверка по URI

Введите URI документа (HTML с CSS или только CSS) для проверки:

Адрес:

► [Дополнительные возможности](#)

Проверить



The W3C validators rely on community support for hosting and development.
[Donate](#) and help us build better tools for a better web.

6042



Примечание: Если вы хотите проверить каскадные таблицы стилей, встроенные в документ (X)HTML, вы должны сначала [проверить на корректность сам документ \(X\)HTML](#).

[О сервисе](#) [Документация](#) [Скачать](#) [Отзывы](#) [Создатели](#)



COPYRIGHT © 1994-2009 W3C® (MIT, ERCIM, KEIO), ALL RIGHTS RESERVED. W3C LIABILITY, TRADEMARK, DOCUMENT USE AND SOFTWARE LICENSING RULES APPLY. YOUR INTERACTIONS WITH THIS SITE ARE IN ACCORDANCE WITH OUR PUBLIC AND MEMBER PRIVACY STATEMENTS.



2. Терминология и синтаксис CSS

Таблица стилей – набор правил **CSS** и комментариев. Таблица выглядит как текст заданного формата и может быть сохранена в файле (с расширением `.css`).

Регистр не важен (обычно – нижний).

В тексте пробельные символы (более одного) и переводы строк игнорируются.

Комментарии ограничивают при помощи `/*` и `*/`.

Терминология и синтаксис CSS

Каждое правило CSS (за исключением так называемых «at-rules») состоит из **селектора** и **блока объявлений**.

Селектор описывает элементы документа, к которым применяются визуальные настройки.

Селекторы могут группироваться через запятую (в таком случае блок объявлений применяется к каждому из них).

Терминология и синтаксис CSS

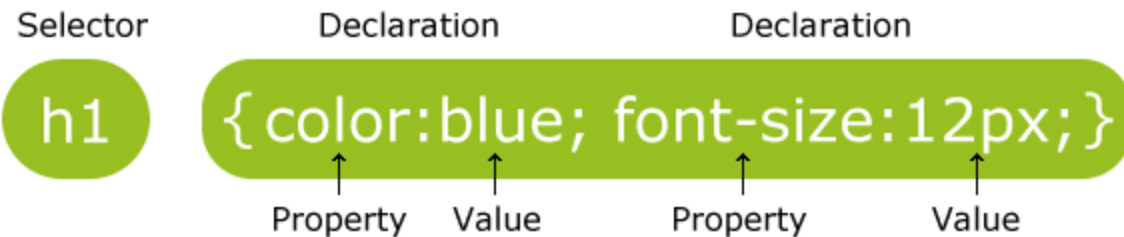
Блок объявлений записывается в фигурных скобках и содержит одно или несколько **объявлений**.

Общий вид объявления следующий (есть нюансы):

свойство : *значение* ;

Пример правила CSS

```
p {  
    color: red;  
    /* This is a single-line comment */  
    text-align: center;  
}
```



Значения CSS-свойств

Значением свойства может быть:

- *Одно значение из фиксированного набора*
- *Число (целое десятичное число или десятичная дробь)*
- *Процентная запись (число + %)*
- *Размер*
- *Цвет*
- *Адрес (URL)*
- *Строка (в одинарных или двойных кавычках)*
- *Временной промежуток*
- *Величина угла*

Значения CSS-свойств

Для некоторых свойств допустимо задавать несколько значений через пробел.

Значение CSS-свойства может быть снабжено модификатором `important` в следующей форме:

свойство: значение **`!important`**;

Этот модификатор управляет **приоритетом** применения значения (подробности будут дальше).

Единицы измерения размера

АБСОЛЮТНЫЕ

- **cm** сантиметры
- **mm** миллиметры
- **in** дюймы
- **pt** пункты, $1\text{pt} = \frac{1}{72}\text{in}$
- **pc** пики, $1\text{pc} = 12\text{pt}$

ОТНОСИТЕЛЬНЫЕ

- **px** пиксели, 1/96 дюйма
- **em** размер шрифта элемента
- **rem** размер шрифта корневого элемента
- **ex** высота символа **x**
- **%** процент от значения у родителя
- **vw, vh** 1/100 ширины и высоты окна

Задание цвета

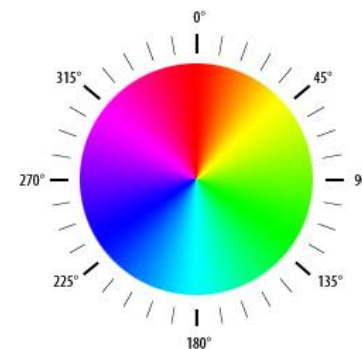
1. Использовать имя: `aqua`, `black`, `blue`, `fuchsia`, `gray`, `green`, `lime`, `maroon`, `navy`, `olive`, `purple`, `red`, `silver`, `teal`, `white`, `yellow` (это базовые цвета, есть ещё).

2. Использовать hex-код: `#RRGGBB` или `#RGB` (=RRGGBB)

3. Использовать функции:

- `rgb(r,g,b)` `rgba(r,g,b,a)`
- `hsl(h,s,l)` `hsla(h,s,l,a)`

Аргументы функций: десятичные числа или проценты.

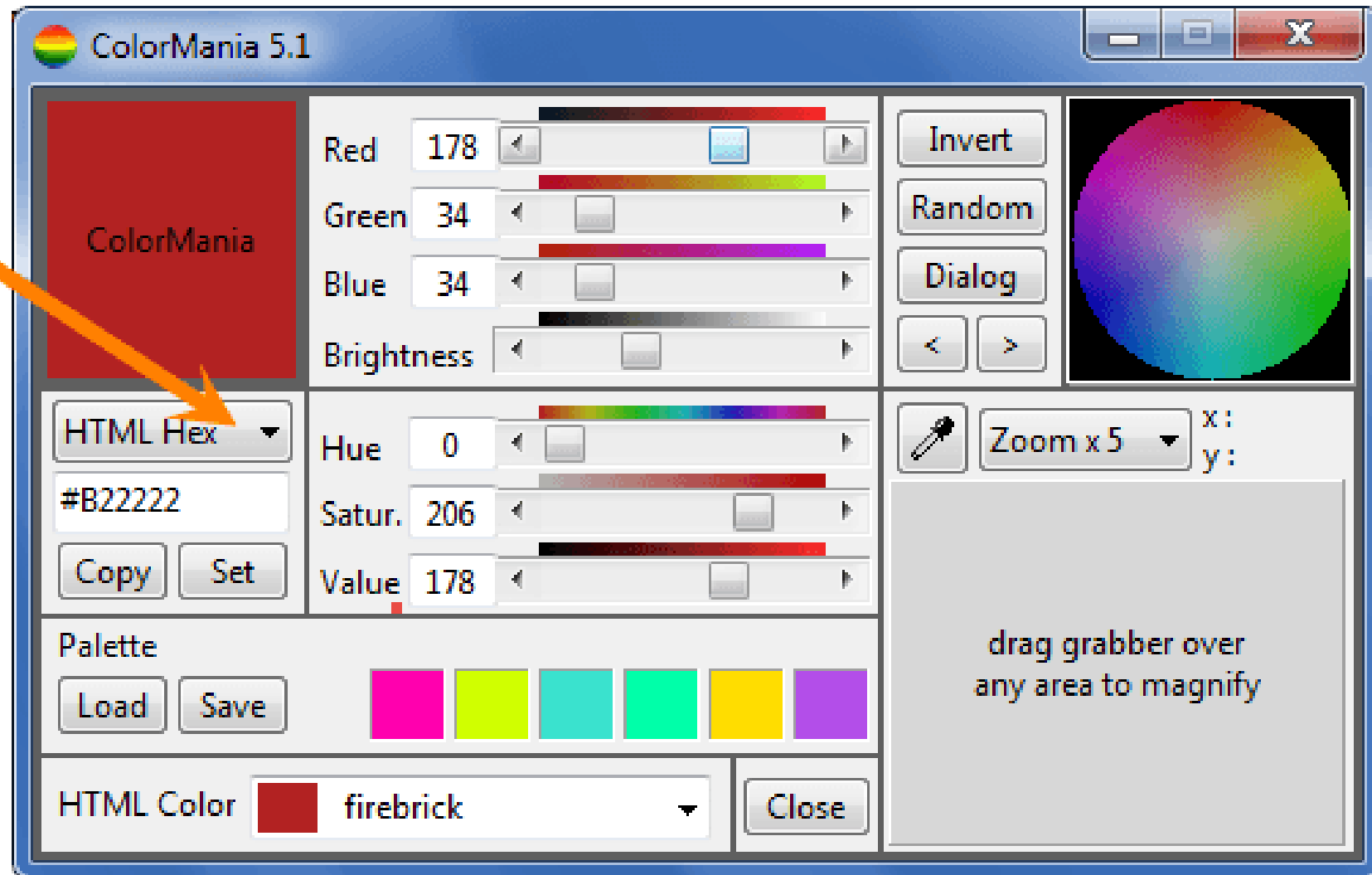


Задание цвета <http://www.colorpicker.com/>



Задание цвета blacksunsoftware.com/colormanial
<https://sovetybloga.ru/colormanial-kak-polzovatsya-kak-uznat-html-kod-cveta/>

**Выбираем
формат
кодировки
цвета**



Задание адреса

В CSS адреса применяются для указания пути к файлу (пример: установка фонового изображения).

Чтобы задать адрес, используется функция `url()`.

Внутри скобок пишется относительный или абсолютный адрес, который можно задавать в необязательных одинарных или двойных кавычках.

Единицы измерения углов, времени

Углы:

`deg` (градусы, 0-360)

`rad` (радианы)

`grad` (градианы, 1 град.=1/100 прямого угла)

`turn` (повороты, 1 поворот = 360 градусов).

Время:

`s` (секунды)

`ms` (миллисекунды).

@charset

Это правило (at-rule) может размещаться **в первой строке** отдельного файла css и задавать кодировку (значение кодировки – в кавычках):

```
@charset "windows-1251";
```

@import

Правило `@import` позволяет импортировать содержимое указанного CSS-файла в текущую стилевую таблицу:

```
@import url("имя файла");
```

```
@import "имя файла";
```

Правило `@import` не разрешается вставлять после любых объявлений кроме `@charset` или другого `@import`.

Подключение CSS к веб-странице – 1

Элемент(ы) **link**, который располагается в **head** и указывает на **внешний** файл, содержащий css.

```
<head>  
    <link rel="stylesheet" href="mystyle.css">  
</head>
```

Подключение CSS к веб-странице – 2

`<head>`

.....

`<style media="all">`

`@import url(style.css);`

`</style>`

`</head>`

Подключение CSS к веб-странице — 3

Контейнер(ы) **style**, который располагается в **head** и **непосредственно** содержит правила CSS и комментарии.

```
<head>
  <style>
    body { color: red; }
  </style>
</head>
```

Подключение CSS к веб-странице – 4

Атрибут **style** у HTML-элемента.

Значением атрибута является строка, в которой перечислены объявления через точку с запятой, **без селектора**.

```
<p style="font-size: 20px; color: green">  
    Test paragraph.  
</p>
```

Подключение CSS к XML-документу

```
<?xml-stylesheet type="text/css" href="style.css"?>
```

Селекторы CSS

Селекторы используются, чтобы найти (выбрать) HTML-элементы на веб-странице, основываясь на имени элемента, идентификаторе, классе, значениях атрибутов и других признаках.

Универсальный селектор

Универсальный селектор выбирает все элементы на странице (включая **body** и **html**). Обозначается звёздочкой:

```
* {  
    border-color: red;  
    border-width: 1px;  
    border-style: solid;  
}
```

Селектор элементов

Селектор элементов выбирает все указанные HTML-элементы на странице. Используется имя элемента:

```
/* выберет все элементы p (абзацы) */  
p {  
    border-width: 1px;  
    border-style: solid;  
}
```

Селектор классов

Селектор классов выбирает HTML-элементы с указанным значением атрибута **class**:

```
/* все элементы, у которых class="main" */  
.main {  
    border-width: 1px;  
    border-style: solid;  
}
```

Селектор идентификаторов

Селектор идентификаторов выбирает HTML-элементы с заданным значением атрибута **id** (теоретически, один):

```
/* элемент(ы), у которых id="pass" */  
#pass {  
    color: red;  
    width: 200px;  
}
```

Селекция по атрибутам

Селектор	Какие элементы выбирает	CSS
[attr]	Имеющие атрибут attr (с любым значением)	2
[attr="val"]	Имеющие атрибут attr со значением val	2
[attr~="val"]	С атрибутом attr , у которого через пробел несколько значений, и одно из этих значений val	2
[attr ="val"]	С атрибутом attr , у которого в значении есть разделители -, и первая часть до - это val	2
[attr^="val"]	Значение атрибута attr начинается с подстроки val	3
[attr\$="val"]	Значение attr заканчивается подстрокой val	3
[attr*="val"]	Значение атрибута attr содержит подстроку val	3

Простой селектор атрибута

Устанавливает стиль для элемента, если задан специфичный атрибут тега. Его значение в данном случае не важно.

[атрибут] { Описание правил стиля }
Селектор[атрибут] { Описание правил стиля }

```
Q {  
    font-style: italic; /* Курсивное начертание */  
    quotes: "\00AB" "\00BB"; /* Меняем вид кавычек в цитате */  
}
```

```
Q[title] {  
    color: maroon; /* Цвет текста */}
```

. . .

<p>Продолжая известный закон Мерфи, который гласит:

<q>Если неприятность может случиться, то она обязательно случится**</q>**, можем ввести свое наблюдение: **<q title="Из законов Фергюссона-Мержевича">**После того, как веб-страница будет корректно отображаться в одном браузере, выяснится, что она неправильно показывается в другом**</q>.</p>**

Продолжая известный закон Мерфи, который гласит: *"Если неприятность может случиться, то она обязательно случится"*, можем ввести свое наблюдение: *"После того, как веб-страница будет корректно отображаться в одном браузере, выяснится, что она неправильно показывается в другом"*.

Атрибут со значением

Устанавливает стиль для элемента в том случае, если задано определённое значение специфического атрибута.

[атрибут="значение"] { Описание правил стиля }

Селектор[атрибут="значение"] { Описание правил стиля }

В первом случае стиль применяется ко всем тегам, которые содержат указанное значение. А во втором — только к определённым селекторам.

```
A[target="_blank"] {  
    background: url(images/blank.png) 0 6px no-repeat;  
/* Параметры фонового рисунка */  
    padding-left: 15px; /* Смещаем текст вправо */  
}
```

. . .

```
<p><a href="1.html">Обычная ссылка</a> |
```

```
<a href="link2" target="_blank">Ссылка в новом
```

```
окне</a></p>
```



Значение атрибута начинается с
определённого текста

Устанавливает стиль для элемента в том случае, если
значение атрибута тега начинается с указанного текста.

[атрибут^="значение"] { Описание правил стиля }

Селектор[атрибут^="значение"] { Описание правил стиля }

```
A[href^="http://"] {
```

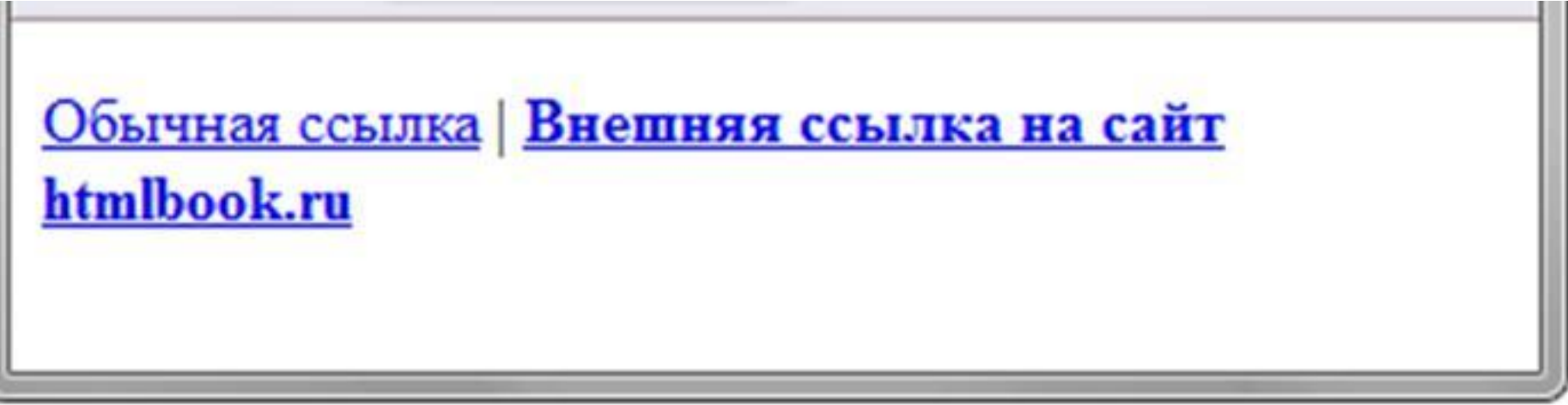
```
    font-weight: bold /* Жирное начертание */
```

```
}
```

. . .

```
<p><a href="1.html">Обычная ссылка</a> |
```

```
<a href="http://htmlbook.ru" target="_blank">Внешняя  
ссылка на сайт htmlbook.ru</a></p>
```



Обычная ссылка | Внешняя ссылка на сайт
htmlbook.ru

Значение атрибута оканчивается определённым текстом

Устанавливает стиль для элемента в том случае, если значение атрибута оканчивается указанным текстом.

[атрибут\$="значение"] { Описание правил стиля }

Селектор[атрибут\$="значение"] { Описание правил стиля }

```
A[href$=".ru"] { /* Если ссылка заканчивается на .ru */  
    background: url(images/ru.png) no-repeat 0 6px; /*  
Добавляем фоновый рисунок */  
    padding-left: 12px; /* Смещаем текст вправо */  
}  
A[href$=".com"] { /* Если ссылка заканчивается на .com */  
    background: url(images/com.png) no-repeat 0 6px;  
    padding-left: 12px;    }
```

. . .

```
<p><a href="http://www.yandex.com">Yandex.Com</a> |  
    <a href="http://www.yandex.ru">Yandex.Ru</a></p>
```

 [Yandex.Com](http://www.yandex.com) |  [Yandex.Ru](http://www.yandex.ru)

Значение атрибута содержит указанный текст

Возможны варианты, когда стиль следует применить к тегу с определённым атрибутом, при этом частью его значения является некоторый текст. При этом точно не известно, в каком месте значения включен данный текст — в начале, середине или конце.

[атрибут*="значение"] { Описание правил стиля }

Селектор[атрибут*="значение"] { Описание правил стиля }

```
[href*="htmlbook"] {  
    background: yellow; /* Желтый цвет фона */  
}
```

. . .

`<p>Теги
HTML |`

`Шаг за
шагом |`

`<a href="http://webimg.ru">Графика для Веб</a></p>`

[Теги HTML](#) | [Шаг за шагом](#) | [Графика для Веб](#)

Одно из нескольких значений атрибута

Некоторые значения атрибутов могут перечисляться через пробел, например имена классов.

Чтобы задать стиль при наличии в списке требуемого значения применяется следующий синтаксис.

[атрибут~="значение"] { Описание правил стиля }

Селектор[атрибут~="значение"] { Описание правил стиля }

```
<style>
```

```
[class~="block"] h3 { color: green; }
```

```
</style>
```

```
. . .
```

```
<div class="block tag">
```

```
<h3>Заголовок</h3>
```

```
</div>
```

В данном примере зелёный цвет текста применяется к селектору H3, если имя класса у слоя задано как block.

Тот же результат *= вместо ~=

Дефис в значении атрибута

В именах идентификаторов и классов разрешено использовать символ дефиса (-), что позволяет создавать значащие значения атрибутов `id` и `class`. Для изменения стиля элементов, в значении которых применяется дефис, следует воспользоваться следующим синтаксисом.

[атрибут]="значение" { Описание правил стиля }

Селектор[атрибут]="значение" { Описание правил стиля }

```
DIV[class | ="block"] {  
  background: #306589;  
  /* Цвет фона */  
  color: #acdb4c;  
  /* Цвет текста */  
  padding: 5px;  
  /* Поля */  
}  
DIV[class | ="block"] A {  
  color: #fff; /* Цвет ссылок */  
}
```

```
<div class="block-menu-therm">  
  <h2>Термины</h2>  
  <div class="content">  
    <ul class="menu">  
      <li><a href="t1.html">Буквица</a></li>  
      <li><a href="t2.html">Выворотка</a></li>  
      <li><a href="t3.html">Выключка</a></li>  
      <li><a href="t4.html">Интерлиньяж</a></li>  
      <li><a href="t5.html">Капитель</a></li>  
      <li><a href="t6.html">Начертание</a></li>  
      <li><a href="t7.html">Отбивка</a></li>  
    </ul>  
  </div>  
</div>
```

Термины

- Буквица
- Выворотка
- Выключка
- Интерлиньяж
- Капитель
- Начертание
- Отбивка

имя класса задано как block-menu-therm, поэтому в стилях используется конструкция |="block", поскольку значение начинается именно с этого слова и в значении встречаются дефисы.

Все перечисленные методы можно комбинировать между собой, определяя стиль для элементов, которые содержат два и более атрибута. В подобных случаях квадратные скобки идут подряд.

```
[атрибут1="значение1"] [атрибут2="значение2"] {  
  Описание правил стиля  
}
```

```
Селектор[атрибут1="значение1"] [атрибут2="значение2"] {  
  Описание правил стиля  
}
```

Комбинации селекторов

Селекторы могут быть скомбинированы путём простого объединения, **без пробелов** (работает как AND):

<code>p.main</code>	элементы <code>p</code> , у которых <code>class="main"</code>
<code>#pass.main</code>	элементы с <code>id="pass"</code> и <code>class="main"</code>
<code>.main.sub</code>	элементы с <code>class="main sub"</code>
<code>p#pass.main</code>	<code>p</code> с <code>id="pass"</code> и <code>class="main"</code>

Комбинации селекторов

Селектор потомков $S1\ S2$ выбирает все элементы по селектору $S2$, которые вложены (на любом уровне) в элементы, выбранные по $S1$:

```
/* любые p, находящиеся внутри любого div */  
div p { color: red; }
```

```
/* элементы с классом main, находящиеся внутри p */  
p .main { color: blue; }
```

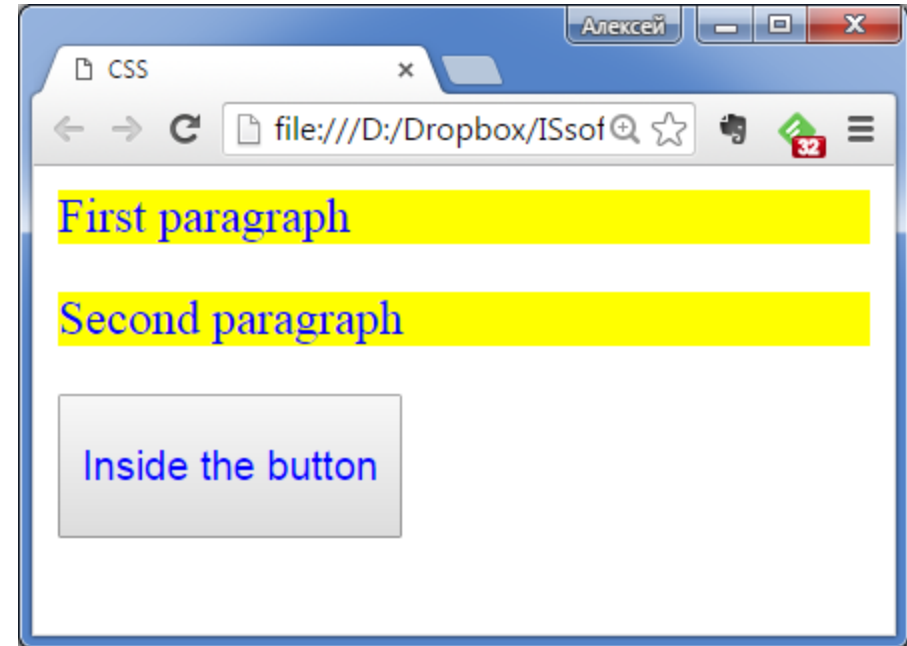

Комбинации селекторов

Селектор дочерних элементов $S1 > S2$ похож на $S1 S2$, но выбирает только **прямых детей** у $S1$:

```
/* p, непосредственно вложенные в div */  
div > p { color: red; }
```

Комбинации селекторов

```
<html>
<head>
  <style>
    div p { color: blue; }
    div > p { background-color: yellow; }
  </style>
</head>
<body>
  <div>
    <p>First paragraph</p>
    <p>Second paragraph</p>
    <button>
      <p>Inside the button</p>
    </button>
  </div>
</body>
</html>
```



Комбинации селекторов

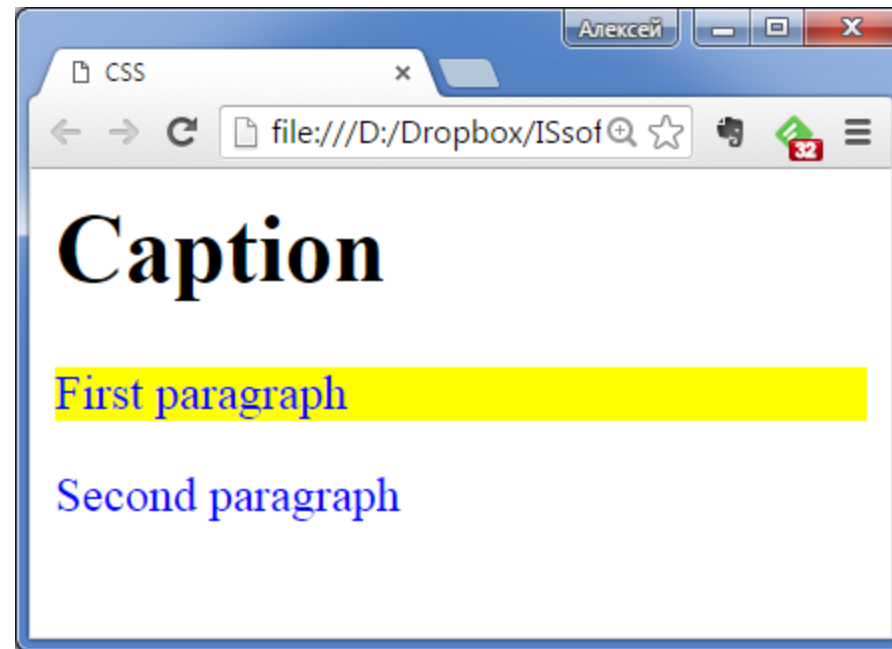
Селектор $S1 + S2$ выбирает элементы по $S2$, которые следуют в документе **непосредственно** за элементами, выбранными по $S1$.

Селектор $S1 \sim S2$ выбирает элементы по $S2$, которые следуют в документе за элементами, выбранными по $S1$ (необязательно непосредственно).

Важно: речь идёт об элементах одного уровня

Комбинации селекторов

```
<html>
<head>
  <title>CSS</title>
  <style>
    h1 + p {background-color: yellow;}
    h1 ~ p {color: blue;}
  </style>
</head>
<body>
  <h1>Caption</h1>
  <p>First paragraph</p>
  <p>Second paragraph</p>
</body>
</html>
```



Псевдоклассы

Псевдоклассы служат для описания

- ❑ динамического состояния элемента, которое изменяется в ответ на действия пользователя (пример: ссылка меняет цвет при наведении курсора);
- ❑ положения элемента в дереве документа (пример: первый потомок).

Псевдоклассы — состояние

Псевдокласс	Что выбирает	CSS
<code>:link</code>	Стиль для ссылки, которая не была посещена	1
<code>:visited</code>	Стиль для ссылки, которая была посещена	1
<code>:hover</code>	Наведён курсор мыши	2
<code>:active</code>	Ссылка нажата, но курсор мыши ещё не отпустили	2
<code>:focus</code>	Элемент находится в фокусе	2

:hover

HTML

<p>Наведите курсор на эту ссылку и увидите, как она становится красной.</p>

CSS

```
a { color: blue; }
```

```
a:hover { color: red; }
```

Наведите курсор на эту ссылку и увидите, как она становится красной.

Наведите курсор на эту ссылку и увидите, как она становится красной.

:visited

HTML

```
<a href="https://www.google.com">Google</a><a  
href="https://twitter.com">Twitter</a><a  
href="https://www.facebook.com">Facebook</a><a  
href="https://www.mozilla.org">Mozilla</a><a  
href="http://marksheet.io/visited.html">MarkSheet</a>
```

CSS

```
a { color: dodgerblue; }  
a:visited { color: rebeccapurple; }
```

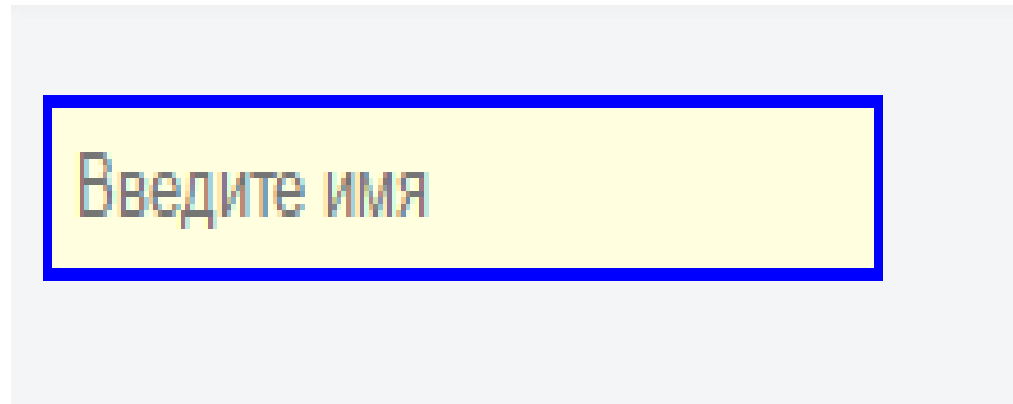
[Google](https://www.google.com) [Twitter](https://twitter.com) [Facebook](https://www.facebook.com) [Mozilla](https://www.mozilla.org) [MarkSheet](http://marksheet.io/visited.html)

:focus

```
.form-input { border: 2px solid grey; padding: 5px; }
```

```
.form-input:focus { background: lightyellow; border-color: blue; outline: none; }
```

```
<input class="form-input" placeholder="Введите имя"> </p>
```



Псевдоклассы – структура

Псевдокласс	Что выбирает	CSS
<code>:first-child</code>	Первый дочерний элемент в контейнере	2
<code>:root</code>	Корневой элемент документа (т.е. html)	3
<code>:last-child</code>	Последний дочерний элемент в контейнере	3
<code>:only-child</code>	Единственный дочерний элемент в контейнере	3
<code>:only-of-type</code>	Единственный дочерний элемент своего типа в контейнере	3
<code>:nth-child(n)</code>	N-ый дочерний элемент	3
<code>:nth-last-child(n)</code>	N-ый дочерний элемент (с конца)	3
<code>:nth-of-type(n)</code>	N-ый дочерний элемент своего типа	3
<code>:nth-last-of-type(n)</code>	N-ый дочерний элемент своего типа (с конца)	3

:first-child и :last-child

HTML

```
<ul>
```

```
  <li>Один</li>
```

```
  <li>Два</li>
```

```
  <li>Три</li>
```

```
  <li>Четыре</li>
```

```
</ul>
```

CSS

```
li:first-child { background: greenyellow; }
```

```
li:last-child { background: lightsalmon; }
```

- Один
- Один
- Два
- Три
- Четыре
- Пять

:nth-child

```
li:nth-child(2) { background: violet; }
```

- Один
- Два
- Три
- Четыре

odd и even

`:nth-child(odd)` нацелится на каждый нечётный элемент;

`:nth-child(even)` нацелится на каждый чётный элемент.

```
li:nth-child(odd) { background: gold; }
```

- Один
- Два
- Три
- Четыре

***Счётчик n

```
li:nth-child(3n) { background: hotpink; }
```

- Один
- Два
- Три
- Четыре
- Пять
- Шесть
- Семь

:nth-child(3n) как «нацелиться на каждый элемент, положение которого делится на 3».

3 умножить на 0 равен нулю;

3 умножить на 1 — третий пункт;

3 умножить на 2 — шестой пункт.

$n + 1$

```
li:nth-child(3n+1) { background: limegreen; }
```

- Один
- Два
- Три
- Четыре
- Пять
- Шесть
- Семь

$3n + 1$ состоит из двух частей:
 $3n$ выбирает каждый **третий** пункт;
 $+1$ **смещает** начало на 1.

Вот как были выполнены вычисления:
 3 умножить на 0 плюс 1 равно 1 ;
 3 умножить на 1 плюс 1 равно 4 ;
 3 умножить на 2 плюс 1 равно 7 .

:first-of-type

```
tr:first-of-type {  
    background: #808990; /* Цвет фона */  
    color: #fff; /* Цвет текст */  
}
```

```
td:first-of-type {  
    background: #CFD6D3; /* Цвет фона */  
}
```

добавление :first-of-type к селектору td устанавливает стиль для всех первых ячеек, поскольку родителем для элемента <td> выступает элемент <tr>.

	1998	1999	2000	2001	2002	2003
Нефть	3	22	34	62	74	57
Золото	4	13	69	72	56	47
Дерево	4	7	73	79	34	86

Псевдоклассы – структура

```
tr:nth-child(4) { color: red; }
```

```
tr:nth-child(2n) { color: red; }
```

```
tr:nth-child(3n+2) { color: red; }
```

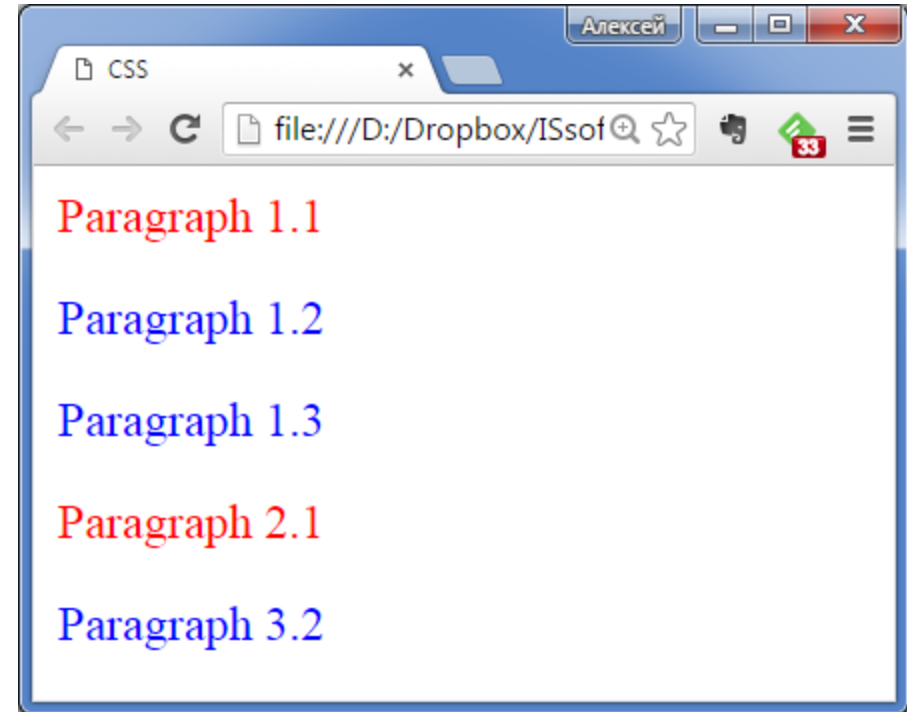
```
tr:nth-child(even) { color: red; }
```

```
tr:nth-child(odd) { color: red; }
```

Псевдоклассы

```
<style>
  p { color: blue; }
  p:first-child { color: red; }
</style>
```

```
<body>
  <div>
    <p>Paragraph 1.1</p>
    <p>Paragraph 1.2</p>
    <p>Paragraph 1.3</p>
  </div>
  <div>
    <p>Paragraph 2.1</p>
    <p>Paragraph 3.2</p>
  </div>
</body>
```



Псевдоклассы

```
<div>
  <p> Paragraph </p>
  <ul>
    <li> Item 1 </li>
    <li> Item 2 </li>
  </ul>
  <ul>
    <li> Item 3 </li>
    <li> item 4 </li>
  </ul>
</div>
```

Выделение **Item 2** – три способа:

```
ul:first-of-type > li:nth-child(2) {
  font-weight: bold;
}
```

```
p + ul li:last-child {
  font-weight: bold;
}
```

```
ul:first-of-type li:nth-last-child(1) {
  font-weight: bold;
}
```

Псевдоклассы – UI (всё в CSS3)

Псевдокласс	Что выбирает
<code>:enabled</code>	Доступные (не заблокированные) элементы формы
<code>:disabled</code>	Заблокированные элементы формы
<code>:checked</code>	Элементы интерфейса (переключатели, флажки), которые находятся в положение «включено»
<code>:default</code>	Элементы, которые установлены по умолчанию в группе похожих
<code>:valid</code> <code>:invalid</code>	Элементы, прошедшие или не прошедшие клиентскую валидацию
<code>:in-range</code> <code>:out-of-range</code>	input со значением в (вне) указанном диапазоне
<code>:required</code> <code>:optional</code>	Элементы, основываясь на наличии атрибута required

:checked

```
input:checked + span {  
  background: #fc0;  
}
```

...

```
<input type="checkbox" name="a2"><span>Windows Vista</span><br>
```

```
<input type="checkbox" name="a3"><span>Windows XP</span><br>
```

...

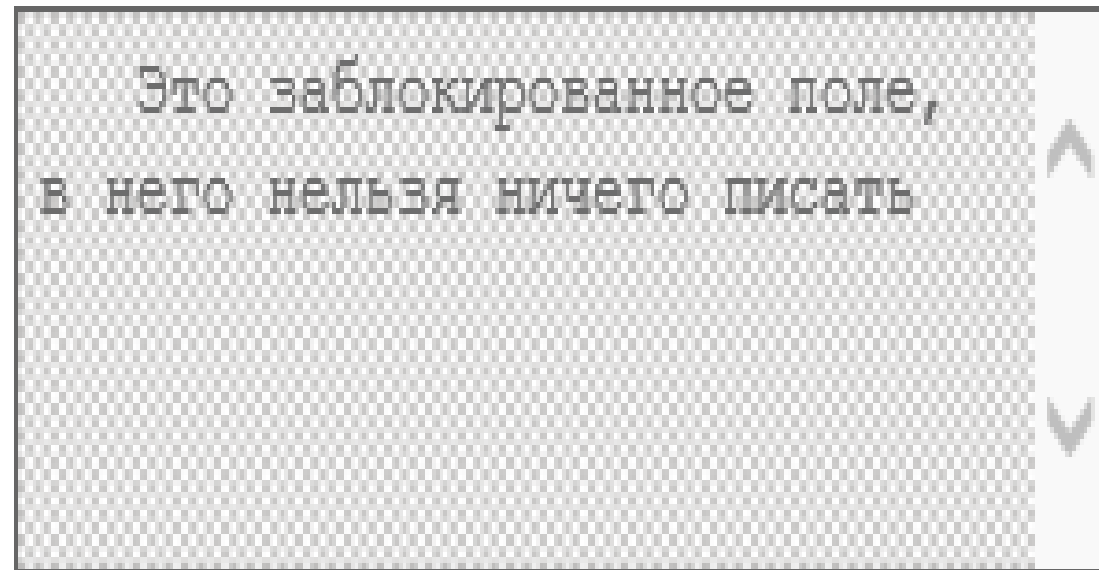
- ☐ Windows 7
- ☒ Windows Vista
- ☐ Windows XP
- ☒ System X
- ☐ Linux
- ☐ Mac OS

:disabled

```
textarea:disabled {  
    background: url(image/dline.png);  
    border: 1px solid #666;  
    padding: 5px; }
```

```
<textarea disabled cols="30" rows="5">
```

Это заблокированное поле, в него нельзя ничего
писать</textarea>



Псевдоклассы – вне категорий

Псевдокласс	Что выбирает
:not(<селектор>)	Элементы, которые не попадают под указанный селектор
:empty	Пустые элементы (не содержат дочерних элементов, текста, пробелов)
:lang(<язык>)	Элементы на основе значения атрибута lang
:target	Целевой элемент (его id задан в адресе) <i>Пример: http://www.w3.org/TR/css3-selectors/#trg</i>

:empty

`<p></p>` является пустым элементом,
а `<p> </p>`, `<p> sp;</p>` или `<p>эге</p>` уже нет.

```
span:empty {
```

```
    background: red;
```

```
    padding: 3px;
```

```
    margin-left: 7px;
```

```
    display: inline-block;
```

```
}
```

Линеарная фактура, в том числе, иллюстрирует фузз, и здесь мы видим ту самую каноническую секвенцию с разнонаправленным шагом отдельных звеньев ■

`<p>`Линеарная фактура, в том числе, иллюстрирует фузз, и здесь мы видим ту самую каноническую секвенцию с разнонаправленным шагом отдельных звеньев`<span></span></p>`

:lang

```
p { font-size: 1.5em; /* Размер текста */ }  
q:lang(de) { quotes: "\201E" "\201C"; }  
q:lang(en) { quotes: "\201C" "\201D"; }  
q:lang(fr), q:lang(ru) { quotes: "\00AB" "\00BB"; }  
...
```

<p>Цитата на французском языке: <q lang="fr">Ce que femme veut,
Dieu le veut</q>.</p>

<p>Цитата на немецком: <q lang="de">Der Mensch, versuche die
Gotter nicht</q>.</p>

<p>Цитата на английском: <q lang="en">To be or not to be</q>.</p>

Цитата на французском языке: «Ce que femme veut, Dieu le veut».

Цитата на немецком: „Der Mensch, versuche die Gotter nicht“.

Цитата на английском: “To be or not to be”.

Псевдоэлементы

Псевдоэлементы позволяют задать стиль элементов, не определённых в дереве элементов документа.

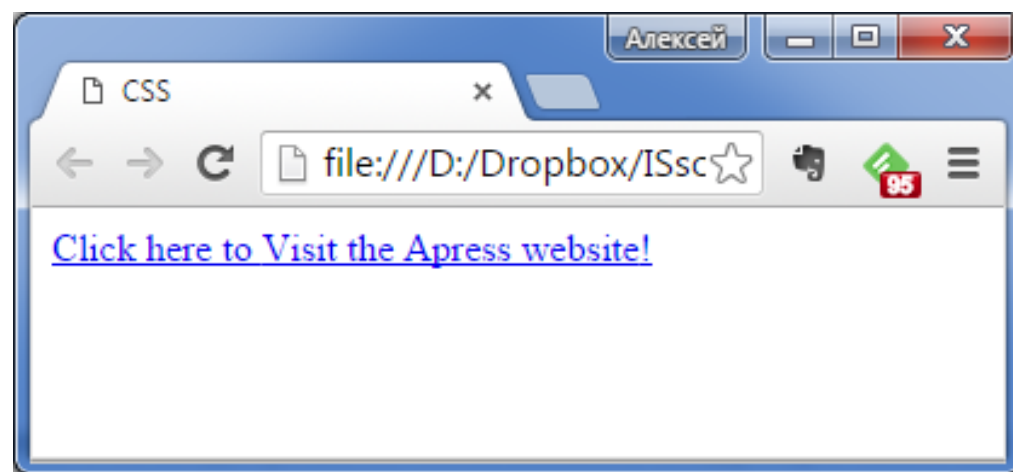
А ещё псевдоэлементы позволяют создать контент.

Псевдоэлементы

Псевдоэлемент	Что выбирает (что делает)	CSS
<code>::first-line</code>	Выбирает первую строку текста	1
<code>::first-letter</code>	Выбирает первую букву текста	1
<code>:before</code>	Используется для вывода желаемого контента перед элементом, к которому он добавляется	2
<code>:after</code>	Используется для вывода желаемого контента после элемента, к которому он добавляется	2
<code>::selection</code>	Применяет стиль к выделенному пользователем фрагменту текста.	3

Псевдоэлементы – пример

```
<html>
<head>
  <title>CSS</title>
  <style type="text/css">
    a:before { content: "Click here to "; }
    a:after { content: "!"; }
  </style>
</head>
<body>
  <a href="http://apress.com">Visit the Apress website</a>
</body>
</html>
```



```
P.new:after {  
  content: " - Новьё!";  
  /* Добавляем после  
  текста абзаца */  
}
```

Ловля льва в пустыне с помощью метода золотого сечения. - Новьё!

Метод ловли льва простым перебором.

• • •

<p class="new">Ловля льва в пустыне с
помощью метода золотого сечения.</p>
<p>Метод ловли льва простым перебором.</p>

```
UL {  
    padding-left: 0; /* Убираем отступ слева */  
    list-style-type: none; /* Прячем маркеры списка */  
}  
LI:before {  
    content: "\20aa "; /* Добавляем перед элементом  
списка СИМВОЛ В ЮНИКОДЕ */  
}
```

☛ Чебурашка

☛ Крокодил Гена

☛ Шапокляк

☛ Крыса Лариса

Символы юникода:

<http://htmlweb.ru/html/symbols.php>


```
p::first-letter {  
  /* Use :first-letter if support for IE 8 or  
  earlier is needed */  
  color: red;  
  font-size: 130%;  
}  
<p>Duis autem vel eum iriure dolor in hendrerit  
in vulputate velit esse molestie consequat.</p>
```

Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat.

```
P::first-line {  
    color: red; /* Красный цвет текста */  
    font-style: italic; /* Курсивное начертание */ }
```

. . .

<p>Интересно, а существует ли способ действительно практичного применения свойства `first-line`? Нет, не такого, чтобы можно было бы показать, что это возможно, а чтобы воистину захватило дух от красоты решения, загорелись глаза от скрытых перспектив, после чего остается только сказать себе, что вот это вот, это самое сделать по-другому, также изящно и эффектно просто невозможно.**</p>**

Интересно, а существует ли способ действительно
практичного применения свойства first-line? Нет, не такого,
чтобы можно было бы показать, что это возможно, а чтобы
воистинухватило дух от красоты решения, загорелись глаза
от скрытых перспектив, после чего остается только сказать
себе, что вот это вот, это самое сделать по-другому, также
изящно и эффектно просто невозможно.

Наследование

Принцип наследования: некоторые свойства CSS, объявленные для элементов-предков, наследуются элементами потомками.

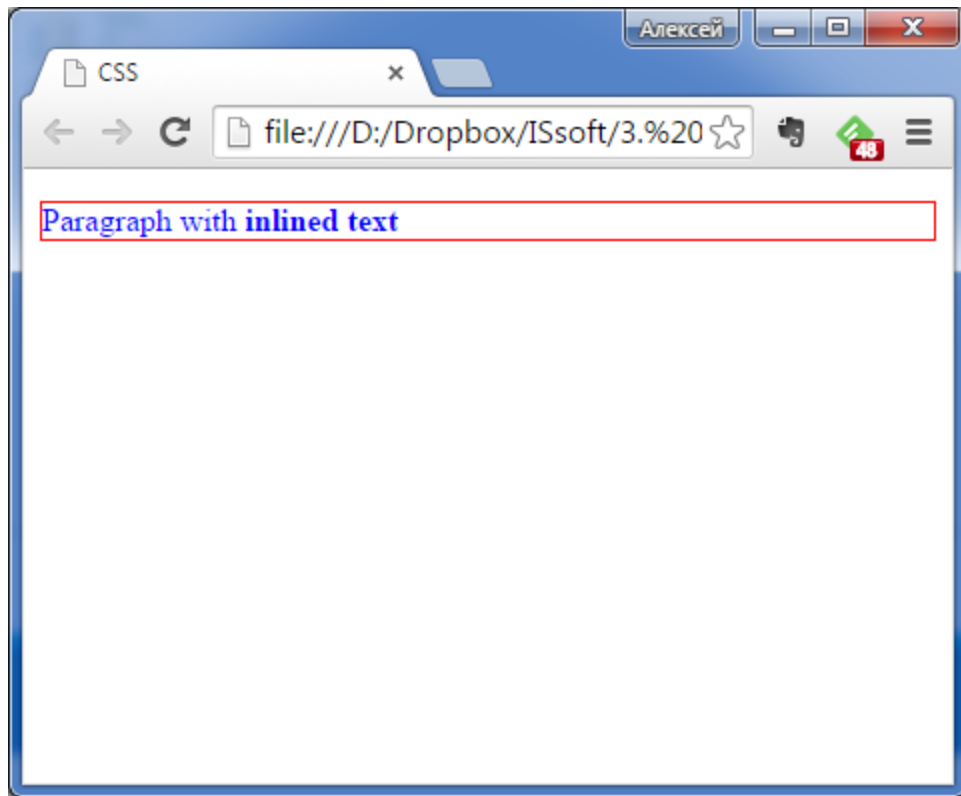
Пример: если задать цвет шрифта для абзаца, всё в абзаце будет иметь этот цвет. Т.е. цвет шрифта – *наследуемое свойство*. А вот рамка вокруг элемента – это не наследуемое свойство.

Наследование

```
<style>
  p {
    border-color: red;
    border-width: 1px;
    border-style: solid;
    color: blue; }
</style>

<body>
  <p>Paragraph with <b>inlined text</b></p>
</body>
```

Наследование



Наследование

Примечание: если свойство является ненаследуемым, то как правило, оно может принимать специальное значение `inherit`.

Это значение сообщает, что необходимо наследовать значение свойства у родительского элемента. Естественно, результат будет заметен только в том случае, если у родителя свойство установлено.

```
div {  
    border: 1px solid green;  
    padding: 10px;  
}  
p {  
    border: inherit;  
    padding: inherit;  
}
```

. . .

`<div><p>`Внимание, запрашиваемая страница не
найдена!`</p></div>`

Внимание, запрашиваемая страница не найдена!

Каскадирование

Введём следующие категории CSS:

1. **Default CSS** — «встроена» в браузер
2. **User CSS** — создаётся и настраивается пользователем сайта (<http://superuser.com/questions/318912/how-to-override-the-css-of-a-site-in-firefox-with-usercontent-css>)
3. **Author CSS** — создаётся автором HTML-страницы

Каскадирование

Согласно принципу каскадирования, порядок применения стилей такой:

1. **Default CSS**
2. **User CSS**
3. **Author CSS**
4. **Author CSS с !important**
5. **User CSS с !important**

Вес правила

Иногда бывает так, что у элемента какое-то свойство CSS установлено в разные значения в разных правилах.

Выяснить реальное значение поможет **вес правила**.

Вес правила

Вес правила – четвёрка чисел вида (at, id, c, e)

- если правило записано в **style**, его вес $(1,0,0,0)$
- id = число селекторов идентификаторов у правила;
- c = число селекторов классов, псевдоклассов, атрибутов;
- e = число селекторов элементов и псевдоэлементов.

Вес правила – примеры

Селектор	at	id	c	e
* {}	0	0	0	0
li {}	0	0	0	1
li:first-line {}	0	0	0	2
ul li {}	0	0	0	2
ul ol+li {}	0	0	0	3
h1 + *[rel=up] {}	0	0	1	1
ul ol li.red {}	0	0	1	3
li.red.level {}	0	0	2	1
#abc {}	0	1	0	0
style=""	1	0	0	0

Вес правила

Весы сравниваем, начиная с самых левых элементов.

В случае равенства смотрим на следующий справа элемент, и так далее.

$$(1, 0, 0, 0) > (0, 0, 2, 1)$$

$$(0, 0, 1, 0) > (0, 0, 0, 4)$$

В итоге применяется то правило, у которого **вес больше**.

Вес правила

```
<head>
  <style>
    #main div { color: black; }
    #sub { color: red; }
  </style>
</head>
<body>
  <!-- какого цвета будет вопросительный знак? -->
  <div id="main">
    <div id="sub">?</div>
  </div>
</body>
```

Вес правила

К `<div id="sub">` подходят оба правила!

`#main div { color: black; }` у этого вес (0,1,0,1)

`#sub { color: red; }` у этого вес (0,1,0,0)

Значит, будет чёрного цвета!


```
p {  
    font-family: arial, helvetica, sans-serif;  
}  
h2 {  
    font-size: 20pt;  
    color: red;  
    background: white;  
}  
.note {  
    color: red;  
    background-color: yellow;  
    font-weight: bold;  
}
```

```
p#paragraph1 {  
    padding-left: 10px;  
}  
a:hover {  
    text-decoration: none;  
}  
#news p {  
    color: blue;  
}  
[type="button"] {  
    background-color: green;  
}
```